E-commerce Chatbot Source Code

Generated 2026-04-28

Included Files

```
chatbot_engine.py
chatbot_features.py
product_catalog.py
analytics.py
category_predictor.py
evaluation.py
generate_research_datasets.py
app.py
chat.py
db.py
train.py
model.py
nltk_utils.py
```

chatbot_engine.py

```
import json
import random

import torch

from model import NeuralNet
from nltk_utils import bag_of_words, tokenize


CONFIDENCE_THRESHOLD = 0.75
PRODUCT_QUESTION_TAGS = {"items", "product_availability", "sizes", "material"}


def get_device():
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")


def load_intents(file_path="intents.json"):
    with open(file_path, "r", encoding="utf-8") as file:
        return json.load(file)


def is_cancel_order_request(user_input):
    normalized = user_input.lower()
    cancel_words = {"cancel", "delete", "remove"}
    return "order" in normalized and any(word in normalized for word in cancel_words)


def is_create_order_request(user_input):
    normalized = user_input.lower()
    create_words = {"create", "place", "make", "add", "buy"}
    return any(word in normalized for word in create_words) and (
        "order" in normalized or "product" in normalized
    )


def is_admin_login_request(user_input):
    return user_input.lower() in {"admin login", "login as admin", "admin"}


def is_admin_logout_request(user_input):
    return user_input.lower() in {"admin logout", "logout"}


def is_update_status_request(user_input):
    normalized = user_input.lower()
    status_words = {"status", "location", "shipped", "delivered", "processing"}
    update_words = {"update", "change", "set", "mark"}
    return (
        "order" in normalized
        and any(word in normalized for word in status_words)
        and any(word in normalized for word in update_words)
    )


def is_recommendation_request(user_input):
    normalized = user_input.lower()
    recommendation_words = {
        "recommend",
        "recommendation",
        "suggest",
        "suggestion",
        "similar",
        "personalized",
        "best match",
        "you think i should buy",
        "what should i buy",
    }
    return any(word in normalized for word in recommendation_words)


def is_product_popularity_request(user_input):
    normalized = user_input.lower()
    popularity_words = {
        "popular product",
        "popular products",
        "product popularity",
        "best selling",
        "best-selling",
```

```
        "top selling",
        "top products",
        "sales analysis",
        "analytics",
    }
    return any(word in normalized for word in popularity_words)


def is_market_basket_request(user_input):
    normalized = user_input.lower()
    basket_words = {
        "market basket",
        "apriori",
        "frequently bought together",
        "bought together",
        "product pairs",
        "basket analysis",
    }
    return any(word in normalized for word in basket_words)


def is_category_prediction_request(user_input):
    normalized = user_input.lower()
    category_words = {
        "predit category",
        "predit",
        "predcit category",
        "predict catagory",
        "predict category",
        "category prediction",
        "what category",
        "which category",
        "classify product",
        "classify this product",
        "product category",
    }
    return any(word in normalized for word in category_words)


class ChatbotEngine:
    def __init__(self, model, all_words, tags, intents, device=None):
        self.model = model
        self.all_words = all_words
        self.tags = tags
        self.intents = intents
        self.device = device or get_device()

    @classmethod
    def from_files(cls, model_path="data.pth", intents_path="intents.json"):
        device = get_device()
        data = torch.load(model_path, map_location=device)
        model = NeuralNet(
            data["input_size"],
            data["hidden_size"],
            data["output_size"],
        ).to(device)
        model.load_state_dict(data["model_state"])
        model.eval()
        return cls(
            model=model,
            all_words=data["all_words"],
            tags=data["tags"],
            intents=load_intents(intents_path),
            device=device,
        )

    def predict(self, sentence):
        tokens = tokenize(sentence)
        features = bag_of_words(tokens, self.all_words)
        features = features.reshape(1, features.shape[0])
        features = torch.from_numpy(features).to(self.device)

        with torch.no_grad():
            output = self.model(features)
            _, predicted = torch.max(output, dim=1)
            probs = torch.softmax(output, dim=1)

        tag = self.tags[predicted.item()]
        probability = probs[0][predicted.item()].item()
        return tag, probability

    def get_response_text(self, tag, context=None):
```

```
for intent in self.intents["intents"]:
    if tag != intent["tag"]:
        continue

    if context:
        for response in intent["responses"]:
            if "{context}" in response:
                return response.format(context=context)

    response = random.choice(intent["responses"])
    if "{context}" in response:
        return response.format(context=tag)
    return response

return "I do not understand..."
```

chatbot_features.py

```
from analytics import (
    format_market_basket_analysis,
    format_product_popularity,
    market_basket_analysis,
    product_popularity,
)
from category_predictor import format_category_prediction, predict_category
from product_catalog import format_recommendation_list, recommend_products


def recommendation_message(user_input, viewed_product_ids=None, cart_product_ids=None):
    recommendations = recommend_products(
        query=user_input,
        viewed_product_ids=viewed_product_ids,
        cart_product_ids=cart_product_ids,
    )
    return format_recommendation_list(recommendations)


def product_popularity_message():
    return format_product_popularity(product_popularity())


def market_basket_message():
    return format_market_basket_analysis(market_basket_analysis())


def category_prediction_message(user_input):
    return format_category_prediction(predict_category(user_input))
```

product_catalog.py

```
import csv
import os
import re
from collections import Counter


PRODUCTS_CSV = "products.csv"
PRODUCT_ID_PATTERN = re.compile(r"\bP\d{3,}\b", re.IGNORECASE)


def load_products(csv_path=PRODUCTS_CSV):
    if not os.path.exists(csv_path):
        return []

    with open(csv_path, mode="r") as file:
        return list(csv.DictReader(file))


def get_product_by_id(product_id, csv_path=PRODUCTS_CSV):
    product_id = (product_id or "").strip().lower()
    for product in load_products(csv_path):
        if product["product_id"].lower() == product_id:
            return product
    return None


def find_product_reference(text, csv_path=PRODUCTS_CSV):
    normalized = (text or "").strip().lower()
    if not normalized:
        return None

    product_id_match = PRODUCT_ID_PATTERN.search(normalized)
    if product_id_match:
        product = get_product_by_id(product_id_match.group(0), csv_path)
        if product:
            return product

    for product in load_products(csv_path):
        name = product["name"].lower()
        if name in normalized or normalized in name:
            return product
    return None


def format_product(product):
    return (
        f"{product['product_id']} - {product['name']} | "
        f"{product['color']} | {product['size']} | "
        f"${product['price']} | Stock: {product['stock']}"
    )


def format_product_list(products):
    if not products:
        return "No products are available right now."
    return "\n".join(format_product(product) for product in products)


def _product_tokens(product):
    text = " ".join(
        [
            product.get("name", ""),
            product.get("category", ""),
            product.get("color", ""),
            product.get("size", ""),
            product.get("material", ""),
            product.get("description", ""),
        ]
    ).lower()
    return set(re.findall(r"[a-z0-9]+", text))


def _price(product):
    try:
        return float(product.get("price") or 0)
    except ValueError:
        return 0
```

```
def _stock(product):
    try:
        return int(product.get("stock") or 0)
    except ValueError:
        return 0


def _similarity_score(candidate, source):
    score = 0
    if candidate.get("category") == source.get("category"):
        score += 4
    if candidate.get("material") == source.get("material"):
        score += 2
    if candidate.get("color") == source.get("color"):
        score += 1.5
    if candidate.get("size") == source.get("size"):
        score += 1

    candidate_tokens = _product_tokens(candidate)
    source_tokens = _product_tokens(source)
    if candidate_tokens or source_tokens:
        overlap = len(candidate_tokens & source_tokens)
        union = len(candidate_tokens | source_tokens)
        score += 2 * (overlap / union)

    price_gap = abs(_price(candidate) - _price(source))
    score += max(0, 1 - (price_gap / 25))
    score += min(_stock(candidate), 20) / 20
    return score


def _preference_profile(products):
    profile = {
        "category": Counter(),
        "color": Counter(),
        "material": Counter(),
        "size": Counter(),
    }
    for product in products:
        for field in profile:
            value = product.get(field)
            if value:
                profile[field][value] += 1
    return profile


def _preference_score(candidate, profile):
    weights = {
        "category": 3,
        "material": 2,
        "color": 1.5,
        "size": 1,
    }
    return sum(
        weights[field] * profile[field][candidate.get(field, "")]
        for field in profile
    )


def recommend_products(
    query=None,
    viewed_product_ids=None,
    cart_product_ids=None,
    limit=3,
    csv_path=PRODUCTS_CSV,
):
    products = load_products(csv_path)
    if not products:
        return []

    product_by_id = {product["product_id"]: product for product in products}
    source_products = []
    explicit_source_ids = set()
    explicit_product = find_product_reference(query, csv_path)
    if explicit_product:
        source_products.append(explicit_product)
        explicit_source_ids.add(explicit_product["product_id"])

    cart_ids = set(cart_product_ids or [])
    if not explicit_product:
        for product_id in cart_product_ids or []:
            product = product_by_id.get(product_id)
```

```
            if product and product not in source_products:
                source_products.append(product)

        for product_id in viewed_product_ids or []:
            product = product_by_id.get(product_id)
            if product and product not in source_products:
                source_products.append(product)

    viewed_ids = set(viewed_product_ids or [])
    source_ids = explicit_source_ids
    if not explicit_product:
        source_ids |= cart_ids
    if not explicit_product and len(viewed_ids) < len(products):
        source_ids |= viewed_ids
    candidates = [product for product in products if product["product_id"] not in source_ids]

    if not candidates:
        candidates = products

    if not source_products:
        ranked = sorted(
            candidates,
            key=lambda product: (
                _stock(product),
                product.get("category") == "Gift Set",
                -_price(product),
            ),
            reverse=True,
        )
        return ranked[:limit]

    profile = _preference_profile(source_products)
    ranked = sorted(
        candidates,
        key=lambda product: (
            max(_similarity_score(product, source) for source in source_products)
            + _preference_score(product, profile),
            _stock(product),
        ),
        reverse=True,
    )
    return ranked[:limit]


def recommendation_reason(product, source_products=None):
    source_products = source_products or []
    if source_products:
        source = source_products[0]
        matching_fields = [
            field
            for field in ("category", "material", "color", "size")
            if product.get(field) == source.get(field)
        ]
        if matching_fields:
            return "matches your interest in " + ", ".join(matching_fields)
    if _stock(product) > 0:
        return "available now and fits common mug preferences"
    return "closest catalog match"


def format_recommendation_list(products):
    if not products:
        return "I do not have enough product data to make recommendations right now."
    lines = ["Recommended products:"]
    for product in products:
        lines.append(f"- {format_product(product)}")
    return "\n".join(lines)
```

analytics.py

```
import csv
from collections import Counter, defaultdict
from itertools import combinations


ORDERS_CSV = "orders.csv"


def load_csv_order_rows(csv_path=ORDERS_CSV):
    with open(csv_path, mode="r") as file:
        return list(csv.DictReader(file))


def load_database_order_rows():
    from db import get_connection

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT
                order_number,
                status,
                current_location,
                expected_delivery_date,
                customer_address,
                customer_name,
                customer_email,
                product_id,
                product_name,
                quantity,
                unit_price,
                total_price
            FROM dbo.orders;
            """
        )
        rows = []
        for row in cursor.fetchall():
            rows.append(
                {
                    "order_number": row.order_number,
                    "status": row.status,
                    "current_location": row.current_location,
                    "expected_delivery_date": row.expected_delivery_date,
                    "customer_address": row.customer_address,
                    "customer_name": row.customer_name,
                    "customer_email": row.customer_email,
                    "product_id": row.product_id,
                    "product_name": row.product_name,
                    "quantity": row.quantity,
                    "unit_price": row.unit_price,
                    "total_price": row.total_price,
                }
            )
        return rows


def load_order_rows(csv_path=ORDERS_CSV, use_database=True):
    if use_database:
        try:
            return load_database_order_rows()
        except Exception:
            pass
    return load_csv_order_rows(csv_path)


def _quantity(row):
    try:
        return int(row.get("quantity") or 1)
    except ValueError:
        return 1


def _total_price(row):
    try:
        return float(row.get("total_price") or 0)
    except ValueError:
        return 0
```

```
def product_popularity(csv_path=ORDERS_CSV, limit=5, use_database=True):
    quantity_counter = Counter()
    revenue_counter = Counter()
    name_by_id = {}

    for row in load_order_rows(csv_path, use_database):
        if row.get("status", "").lower() == "cancelled":
            continue

        product_id = row.get("product_id")
        if not product_id:
            continue

        name_by_id[product_id] = row.get("product_name") or product_id
        quantity_counter[product_id] += _quantity(row)
        revenue_counter[product_id] += _total_price(row)

    return [
        {
            "product_id": product_id,
            "product_name": name_by_id.get(product_id, product_id),
            "units_sold": units_sold,
            "revenue": revenue_counter[product_id],
        }
        for product_id, units_sold in quantity_counter.most_common(limit)
    ]


def format_product_popularity(rows):
    if not rows:
        return "No product sales data is available yet."

    lines = ["Product popularity analysis from saved orders:"]
    for index, row in enumerate(rows, start=1):
        lines.append(
            f"{index}. {row['product_id']} - {row['product_name']} | "
            f"Units sold: {row['units_sold']} | Revenue: ${row['revenue']:.2f}"
        )
    return "\n".join(lines)


def market_basket_analysis(
    csv_path=ORDERS_CSV,
    min_support=2,
    limit=5,
    use_database=True,
):
    baskets = defaultdict(set)
    product_names = {}

    for row in load_order_rows(csv_path, use_database):
        if row.get("status", "").lower() == "cancelled":
            continue

        product_id = row.get("product_id")
        customer_email = row.get("customer_email")
        if not product_id or not customer_email:
            continue

        baskets[customer_email].add(product_id)
        product_names[product_id] = row.get("product_name") or product_id

    pair_counter = Counter()
    item_counter = Counter()
    basket_count = 0
    for basket in baskets.values():
        if len(basket) < 2:
            continue
        basket_count += 1
        for product_id in basket:
            item_counter[product_id] += 1
        for pair in combinations(sorted(basket), 2):
            pair_counter[pair] += 1

    results = []
    for pair, support_count in pair_counter.most_common():
        if support_count < min_support:
            continue
        left, right = pair
        support = support_count / max(basket_count, 1)
        left_support = item_counter[left] / max(basket_count, 1)
```

```
        right_support = item_counter[right] / max(basket_count, 1)
        confidence_left_to_right = support_count / max(item_counter[left], 1)
        confidence_right_to_left = support_count / max(item_counter[right], 1)
        lift_left_to_right = confidence_left_to_right / max(right_support, 0.000001)
        lift_right_to_left = confidence_right_to_left / max(left_support, 0.000001)
        results.append(
            {
                "pair": pair,
                "names": tuple(product_names[product_id] for product_id in pair),
                "support_count": support_count,
                "support": support,
                "confidence_left_to_right": confidence_left_to_right,
                "confidence_right_to_left": confidence_right_to_left,
                "lift_left_to_right": lift_left_to_right,
                "lift_right_to_left": lift_right_to_left,
            }
        )
        if len(results) >= limit:
            break

    return results


def format_market_basket_analysis(rows):
    if not rows:
        return "No frequent product pairs were found yet."

    lines = ["Market basket analysis from saved orders:"]
    for row in rows:
        left, right = row["pair"]
        left_name, right_name = row["names"]
        lines.append(
            f"- {left} ({left_name}) + {right} ({right_name}) | "
            f"Support count: {row['support_count']} | "
            f"Support: {row['support']:.0%} | "
            f"Confidence {left}->{right}: {row['confidence_left_to_right']:.0%} | "
            f"Lift: {row['lift_left_to_right']:.2f}"
        )
    return "\n".join(lines)
```

category_predictor.py

```
import math
import re
from collections import Counter, defaultdict

from product_catalog import find_product_reference, load_products


TOKEN_PATTERN = re.compile(r"[a-z0-9]+")


def tokenize_text(text):
    return TOKEN_PATTERN.findall((text or "").lower())


def product_text(product):
    return " ".join(
        [
            product.get("name", ""),
            product.get("color", ""),
            product.get("size", ""),
            product.get("material", ""),
            product.get("description", ""),
        ]
    )


class ProductCategoryPredictor:
    def __init__(self):
        self.category_counts = Counter()
        self.token_counts = defaultdict(Counter)
        self.total_tokens = Counter()
        self.vocabulary = set()
        self.total_examples = 0

    def fit(self, products):
        for product in products:
            category = product.get("category")
            if not category:
                continue

            self.category_counts[category] += 1
            self.total_examples += 1
            for token in tokenize_text(product_text(product)):
                self.token_counts[category][token] += 1
                self.total_tokens[category] += 1
                self.vocabulary.add(token)
        return self

    def predict_proba(self, text):
        tokens = tokenize_text(text)
        if not self.category_counts:
            return {}

        vocabulary_size = max(len(self.vocabulary), 1)
        scores = {}
        for category, category_count in self.category_counts.items():
            score = math.log(category_count / self.total_examples)
            denominator = self.total_tokens[category] + vocabulary_size
            for token in tokens:
                numerator = self.token_counts[category][token] + 1
                score += math.log(numerator / denominator)
            scores[category] = score

        max_score = max(scores.values())
        exp_scores = {
            category: math.exp(score - max_score)
            for category, score in scores.items()
        }
        total = sum(exp_scores.values())
        return {
            category: score / total
            for category, score in exp_scores.items()
        }

    def predict(self, text):
        probabilities = self.predict_proba(text)
        if not probabilities:
            return None, 0
        category, probability = max(
```

```
            probabilities.items(),
            key=lambda item: item[1],
        )
        return category, probability


def train_category_predictor(csv_path="products.csv"):
    products = load_products(csv_path)
    return ProductCategoryPredictor().fit(products)


def predict_category(query, csv_path="products.csv"):
    product = find_product_reference(query, csv_path)
    text = product_text(product) if product else query
    predictor = train_category_predictor(csv_path)
    category, probability = predictor.predict(text)
    return {
        "category": category,
        "confidence": probability,
        "matched_product": product,
    }


def format_category_prediction(result):
    if not result["category"]:
        return "I do not have enough product data to predict a category."

    source = ""
    if result.get("matched_product"):
        product = result["matched_product"]
        source = f" for {product['product_id']} - {product['name']}"

    return (
        f"Predicted category{source}: {result['category']} "
        f"(confidence: {result['confidence'] * 100:.2f}%)."
    )


def evaluate_category_predictor(csv_path="products.csv"):
    products = load_products(csv_path)
    if not products:
        return {
            "total": 0,
            "correct": 0,
            "accuracy": 0,
            "results": [],
        }

    predictor = train_category_predictor(csv_path)
    results = []
    correct = 0
    for product in products:
        predicted_category, confidence = predictor.predict(product_text(product))
        expected_category = product.get("category")
        is_correct = predicted_category == expected_category
        if is_correct:
            correct += 1
        results.append(
            {
                "product_id": product.get("product_id"),
                "name": product.get("name"),
                "expected": expected_category,
                "predicted": predicted_category,
                "confidence": confidence,
                "correct": is_correct,
            }
        )

    total = len(products)
    return {
        "total": total,
        "correct": correct,
        "accuracy": correct / total if total else 0,
        "results": results,
    }
```

evaluation.py

```
import csv
import statistics
import time

from category_predictor import evaluate_category_predictor
from chatbot_engine import ChatbotEngine
from product_catalog import recommend_products


SURVEY_CSV = "evaluation_survey.csv"
BOT_NAME = "Lyla"


RECOMMENDATION_TEST_CASES = [
    {
        "query": "Recommend something like P001",
        "relevant": {"P002", "P003", "P006", "P007", "P008"},
    },
    {
        "query": "Recommend something like P002",
        "relevant": {"P001", "P003", "P006", "P007", "P008"},
    },
    {
        "query": "Recommend something like P004",
        "relevant": {"P011", "P012"},
    },
    {
        "query": "Recommend something like P005",
        "relevant": {"P001", "P002", "P003", "P015"},
    },
]


def percentage(value):
    return f"{value * 100:.2f}%"


def evaluate_chatbot_accuracy(engine):
    total = 0
    correct = 0
    total_time = 0
    wrong_predictions = []

    for intent in engine.intents["intents"]:
        expected_tag = intent["tag"]
        for query in intent.get("queries", []):
            start_time = time.perf_counter()
            predicted_tag, probability = engine.predict(query)
            total_time += time.perf_counter() - start_time

            total += 1
            if predicted_tag == expected_tag:
                correct += 1
            else:
                wrong_predictions.append(
                    {
                        "query": query,
                        "expected": expected_tag,
                        "predicted": predicted_tag,
                        "probability": probability,
                    }
                )

    accuracy = correct / total if total else 0
    average_response_time = total_time / total if total else 0
    return {
        "total_queries": total,
        "correct": correct,
        "wrong": total - correct,
        "accuracy": accuracy,
        "average_response_time": average_response_time,
        "wrong_predictions": wrong_predictions,
    }


def _precision_at_k(recommended_ids, relevant_ids, k):
    top_k = recommended_ids[:k]
    if not top_k:
        return 0
```

```
    hits = sum(1 for product_id in top_k if product_id in relevant_ids)
    return hits / len(top_k)


def _reciprocal_rank(recommended_ids, relevant_ids):
    for index, product_id in enumerate(recommended_ids, start=1):
        if product_id in relevant_ids:
            return 1 / index
    return 0


def evaluate_recommendation_quality(k=3):
    case_results = []
    precision_scores = []
    hit_scores = []
    reciprocal_ranks = []

    for case in RECOMMENDATION_TEST_CASES:
        recommendations = recommend_products(query=case["query"], limit=k)
        recommended_ids = [product["product_id"] for product in recommendations]
        relevant_ids = case["relevant"]

        precision = _precision_at_k(recommended_ids, relevant_ids, k)
        hit = any(product_id in relevant_ids for product_id in recommended_ids)
        reciprocal_rank = _reciprocal_rank(recommended_ids, relevant_ids)

        precision_scores.append(precision)
        hit_scores.append(1 if hit else 0)
        reciprocal_ranks.append(reciprocal_rank)
        case_results.append(
            {
                "query": case["query"],
                "recommended": recommended_ids,
                "relevant": sorted(relevant_ids),
                "precision_at_k": precision,
                "hit": hit,
                "reciprocal_rank": reciprocal_rank,
            }
        )

    return {
        "k": k,
        "mean_precision_at_k": statistics.mean(precision_scores)
        if precision_scores
        else 0,
        "hit_rate": statistics.mean(hit_scores) if hit_scores else 0,
        "mean_reciprocal_rank": statistics.mean(reciprocal_ranks)
        if reciprocal_ranks
        else 0,
        "case_results": case_results,
    }


def evaluate_recommendation_ab_test(k=3):
    with_recommendations = evaluate_recommendation_quality(k)
    return {
        "without_recommendation_module": {
            "task_completion_rate": 0,
            "mean_precision_at_k": 0,
            "hit_rate": 0,
        },
        "with_recommendation_module": {
            "task_completion_rate": 1,
            "mean_precision_at_k": with_recommendations["mean_precision_at_k"],
            "hit_rate": with_recommendations["hit_rate"],
        },
    }


def summarize_user_satisfaction(csv_path=SURVEY_CSV):
    with open(csv_path, mode="r") as file:
        rows = list(csv.DictReader(file))

    rating_fields = [
        "chatbot_accuracy",
        "rating_recommendations",
        "rating_speed",
        "rating_overall",
    ]
    summary = {"responses": len(rows)}
    for field in rating_fields:
        values = [int(row[field]) for row in rows if row.get(field)]
```

```
        summary[field] = statistics.mean(values) if values else 0
    return summary


def print_chatbot_accuracy(metrics):
    print("Chatbot response accuracy")
    print(f"- Test queries: {metrics['total_queries']}")
    print(f"- Correct predictions: {metrics['correct']}")
    print(f"- Wrong predictions: {metrics['wrong']}")
    print(f"- Accuracy: {percentage(metrics['accuracy'])}")
    print(f"- Average prediction time: {metrics['average_response_time']:.4f} seconds")
    if metrics["wrong_predictions"]:
        print("- Sample wrong predictions:")
        for item in metrics["wrong_predictions"][:5]:
            print(
                f"  {item['query']} | expected={item['expected']} | "
                f"predicted={item['predicted']} | probability={item['probability']:.2f}"
            )


def print_recommendation_quality(metrics):
    print("\nRecommendation quality")
    print(f"- Precision@{metrics['k']}: {percentage(metrics['mean_precision_at_k'])}")
    print(f"- Hit rate: {percentage(metrics['hit_rate'])}")
    print(f"- Mean reciprocal rank: {metrics['mean_reciprocal_rank']:.2f}")
    for case in metrics["case_results"]:
        print(
            f"- {case['query']} -> recommended={case['recommended']} | "
            f"Precision@{metrics['k']}={percentage(case['precision_at_k'])}"
        )


def print_ab_test(metrics):
    without_module = metrics["without_recommendation_module"]
    with_module = metrics["with_recommendation_module"]
    print("\nChatbot comparison: without vs with recommendation module")
    print(
        "- Without recommendation module | "
        f"Task completion: {percentage(without_module['task_completion_rate'])} | "
        f"Precision@3: {percentage(without_module['mean_precision_at_k'])} | "
        f"Hit rate: {percentage(without_module['hit_rate'])}"
    )
    print(
        "- With recommendation module | "
        f"Task completion: {percentage(with_module['task_completion_rate'])} | "
        f"Precision@3: {percentage(with_module['mean_precision_at_k'])} | "
        f"Hit rate: {percentage(with_module['hit_rate'])}"
    )


def print_satisfaction(summary):
    print("\nUser satisfaction survey")
    print(f"- Responses: {summary['responses']}")
    print(f"- Chatbot accuracy rating: {summary['chatbot_accuracy']:.2f}/5")
    print(f"- Recommendation rating: {summary['rating_recommendations']:.2f}/5")
    print(f"- Speed rating: {summary['rating_speed']:.2f}/5")
    print(f"- Overall rating: {summary['rating_overall']:.2f}/5")


def print_category_prediction(metrics):
    print("\nProduct category prediction")
    print(f"- Test products: {metrics['total']}")
    print(f"- Correct predictions: {metrics['correct']}")
    print(f"- Accuracy: {percentage(metrics['accuracy'])}")
    for result in metrics["results"]:
        print(
            f"- {result['product_id']} {result['name']} | "
            f"expected={result['expected']} | predicted={result['predicted']} | "
            f"confidence={percentage(result['confidence'])}"
        )


if __name__ == "__main__":
    engine = ChatbotEngine.from_files("data.pth", "intents.json")

    accuracy_metrics = evaluate_chatbot_accuracy(engine)
    recommendation_metrics = evaluate_recommendation_quality(k=3)
    comparison_metrics = evaluate_recommendation_ab_test(k=3)
    satisfaction_summary = summarize_user_satisfaction()
    category_metrics = evaluate_category_predictor()

    print_chatbot_accuracy(accuracy_metrics)
```

```
print_recommendation_quality(recommendation_metrics)
print_ab_test(comparison_metrics)
print_satisfaction(satisfaction_summary)
print_category_prediction(category_metrics)
```

generate_research_datasets.py

```
import csv
import random
from datetime import date, timedelta


random.seed(108)


PRODUCTS = [
    ("P001", "Classic Red Mug", "Coffee Mug", "Red", "350ml", "Ceramic", 12.99, 48, "Durable
everyday ceramic mug in red"),
    ("P002", "Classic White Mug", "Coffee Mug", "White", "350ml", "Ceramic", 11.99, 62, "Minimal
 white ceramic mug for coffee or tea"),
    ("P003", "Classic Blue Mug", "Coffee Mug", "Blue", "350ml", "Ceramic", 12.99, 55, "Blue
ceramic mug with dishwasher-safe finish"),
    ("P004", "Large Travel Mug", "Travel Mug", "Black", "500ml", "Stainless Steel", 18.99, 35,
"Insulated travel mug for hot drinks"),
    ("P005", "Gift Mug Set", "Gift Set", "Mixed", "3 x 350ml", "Ceramic", 34.99, 28, "Three-
piece mug gift set with red white and blue mugs"),
    ("P006", "Matte Black Mug", "Coffee Mug", "Black", "400ml", "Ceramic", 14.99, 41, "Modern
matte ceramic mug for coffee and tea"),
    ("P007", "Pastel Pink Mug", "Coffee Mug", "Pink", "300ml", "Ceramic", 13.49, 34, "Small
pastel ceramic mug with smooth finish"),
    ("P008", "Office Grey Mug", "Coffee Mug", "Grey", "350ml", "Ceramic", 10.99, 57, "Neutral
office mug for daily use"),
    ("P009", "Glass Latte Mug", "Latte Mug", "Clear", "450ml", "Glass", 15.99, 27, "Clear glass
mug for latte and layered coffee"),
    ("P010", "Espresso Cup Pair", "Espresso Cup", "White", "2 x 90ml", "Porcelain", 16.99, 30,
"Pair of porcelain espresso cups"),
    ("P011", "Thermal Steel Mug", "Travel Mug", "Silver", "480ml", "Stainless Steel", 19.99, 32,
 "Thermal steel mug with spill-resistant lid"),
    ("P012", "Commuter Mug", "Travel Mug", "Navy", "520ml", "Stainless Steel", 21.99, 26, "Large
 commuter mug for hot drinks on the go"),
    ("P013", "Eco Bamboo Mug", "Eco Mug", "Beige", "400ml", "Bamboo Fiber", 17.49, 24, "Reusable
 bamboo fiber mug with eco-friendly material"),
    ("P014", "Reusable Glass Cup", "Eco Mug", "Clear", "360ml", "Glass", 14.49, 38, "Reusable
glass cup for hot and cold drinks"),
    ("P015", "Holiday Gift Mug Set", "Gift Set", "Mixed", "4 x 300ml", "Ceramic", 39.99, 18,
"Seasonal gift set with four decorated mugs"),
    ("P016", "Minimal Cream Mug", "Coffee Mug", "Cream", "350ml", "Ceramic", 12.49, 44, "Simple
cream ceramic mug for home use"),
    ("P017", "Forest Green Mug", "Coffee Mug", "Green", "380ml", "Ceramic", 13.99, 36, "Green
ceramic mug inspired by outdoor colors"),
    ("P018", "Speckled Stone Mug", "Coffee Mug", "Stone", "400ml", "Stoneware", 15.49, 29,
"Speckled stoneware mug with handmade style finish"),
    ("P019", "Copper Travel Tumbler", "Travel Mug", "Copper", "500ml", "Stainless Steel", 22.49,
 21, "Metallic insulated tumbler for commuting"),
    ("P020", "Kids Small Mug", "Coffee Mug", "Yellow", "250ml", "Ceramic", 9.99, 40, "Small
bright mug suitable for children"),
    ("P021", "Double Wall Glass Mug", "Latte Mug", "Clear", "320ml", "Glass", 18.49, 25, "Double
 wall glass mug that keeps drinks warm"),
    ("P022", "Corporate Logo Mug", "Custom Mug", "White", "350ml", "Ceramic", 13.99, 50, "White
ceramic mug suitable for custom logo printing"),
    ("P023", "Photo Print Mug", "Custom Mug", "White", "350ml", "Ceramic", 15.99, 46,
"Personalized mug for photo printing"),
    ("P024", "Name Engraved Mug", "Custom Mug", "Black", "400ml", "Ceramic", 16.49, 31, "Custom
mug with name engraving option"),
    ("P025", "Premium Porcelain Mug", "Coffee Mug", "White", "360ml", "Porcelain", 18.99, 33,
"Premium porcelain mug for formal settings"),
    ("P026", "Stackable Espresso Set", "Espresso Cup", "Mixed", "6 x 80ml", "Porcelain", 29.99,
19, "Stackable espresso cups for kitchen storage"),
    ("P027", "Camping Enamel Mug", "Outdoor Mug", "Blue", "420ml", "Enamel", 12.49, 37,
"Lightweight enamel mug for camping"),
    ("P028", "Mountain Enamel Mug", "Outdoor Mug", "Green", "420ml", "Enamel", 12.99, 35,
"Outdoor enamel mug with mountain design"),
    ("P029", "Smart Temperature Mug", "Smart Mug", "Black", "400ml", "Stainless Steel", 54.99,
12, "Smart mug with temperature control"),
    ("P030", "Heated Desk Mug", "Smart Mug", "Grey", "380ml", "Stainless Steel", 49.99, 14,
"Desk mug with heating base compatibility"),
    ("P031", "Floral Gift Mug", "Gift Set", "Pink", "350ml", "Ceramic", 19.99, 23, "Gift-ready
floral ceramic mug"),
    ("P032", "Birthday Mug Box", "Gift Set", "Mixed", "2 x 350ml", "Ceramic", 27.99, 20,
"Birthday themed mug gift box"),
    ("P033", "Large Soup Mug", "Soup Mug", "Red", "600ml", "Ceramic", 16.99, 26, "Large handled
mug suitable for soup"),
    ("P034", "Breakfast Bowl Mug", "Soup Mug", "White", "650ml", "Ceramic", 17.49, 24, "Wide mug
 for breakfast or soup"),
    ("P035", "Reusable Coffee Cup", "Eco Mug", "Green", "360ml", "Recycled Plastic", 11.99, 52,
"Lightweight reusable coffee cup"),
```

```
    ("P036", "Cork Grip Eco Cup", "Eco Mug", "Brown", "380ml", "Bamboo Fiber", 16.49, 30, "Eco
cup with cork grip sleeve"),
    ("P037", "Luxury Gold Mug", "Coffee Mug", "Gold", "330ml", "Porcelain", 24.99, 16, "Gold
accented porcelain mug"),
    ("P038", "Marble Ceramic Mug", "Coffee Mug", "White", "380ml", "Ceramic", 17.99, 28, "Marble
 pattern ceramic mug"),
    ("P039", "Tall Latte Glass", "Latte Mug", "Clear", "500ml", "Glass", 16.99, 29, "Tall glass
mug for latte and iced coffee"),
    ("P040", "Reusable Straw Cup", "Eco Mug", "Clear", "600ml", "Glass", 18.49, 22, "Glass cup
with reusable straw and lid"),
    ("P041", "Mini Espresso Cup", "Espresso Cup", "Black", "70ml", "Porcelain", 8.99, 43, "Small
 black espresso cup"),
    ("P042", "Color Espresso Set", "Espresso Cup", "Mixed", "4 x 90ml", "Porcelain", 24.49, 21,
"Colorful espresso cup set"),
    ("P043", "Slim Travel Cup", "Travel Mug", "White", "450ml", "Stainless Steel", 20.49, 27,
"Slim insulated travel cup"),
    ("P044", "Grip Travel Mug", "Travel Mug", "Red", "520ml", "Stainless Steel", 23.99, 18,
"Large travel mug with grip texture"),
    ("P045", "Student Budget Mug", "Coffee Mug", "Blue", "320ml", "Ceramic", 7.99, 70,
"Affordable ceramic mug for students"),
    ("P046", "Teacher Gift Mug", "Gift Set", "White", "350ml", "Ceramic", 18.99, 26, "Gift mug
for teachers with message print"),
    ("P047", "Office Starter Pack", "Gift Set", "Mixed", "6 x 350ml", "Ceramic", 59.99, 11,
"Bulk office mug starter pack"),
    ("P048", "Cafe Style Cappuccino Cup", "Latte Mug", "White", "280ml", "Porcelain", 13.99, 34,
 "Cafe style cup for cappuccino"),
    ("P049", "Insulated Camping Mug", "Outdoor Mug", "Black", "450ml", "Stainless Steel", 17.49,
 25, "Insulated outdoor mug for camping"),
    ("P050", "Adventure Enamel Set", "Outdoor Mug", "Mixed", "4 x 420ml", "Enamel", 36.99, 17,
"Outdoor enamel mug set for groups"),
    ("P051", "Ceramic Tea Mug", "Tea Mug", "Green", "450ml", "Ceramic", 14.99, 32, "Large
ceramic mug for tea"),
    ("P052", "Infuser Tea Mug", "Tea Mug", "White", "420ml", "Ceramic", 19.49, 20, "Tea mug with
 removable infuser"),
    ("P053", "Glass Tea Cup", "Tea Mug", "Clear", "350ml", "Glass", 13.49, 36, "Clear glass cup
for tea"),
    ("P054", "Personalized Couple Mug Set", "Custom Mug", "Mixed", "2 x 350ml", "Ceramic",
31.99, 18, "Personalized couple mug set"),
    ("P055", "Photo Travel Mug", "Custom Mug", "Silver", "470ml", "Stainless Steel", 27.99, 15,
"Travel mug with photo customization"),
    ("P056", "Smart App Mug", "Smart Mug", "Silver", "410ml", "Stainless Steel", 64.99, 10,
"Smart mug controlled by mobile app"),
    ("P057", "Minimal Black Espresso Pair", "Espresso Cup", "Black", "2 x 90ml", "Porcelain",
18.49, 24, "Pair of minimalist black espresso cups"),
    ("P058", "Rustic Stoneware Mug", "Coffee Mug", "Brown", "420ml", "Stoneware", 16.99, 27,
"Rustic stoneware mug with warm finish"),
    ("P059", "Large Office Mug", "Coffee Mug", "Navy", "500ml", "Ceramic", 15.99, 39, "Large
ceramic mug for long work sessions"),
    ("P060", "Compact Travel Cup", "Travel Mug", "Grey", "350ml", "Stainless Steel", 16.49, 33,
"Compact travel cup for small bags"),
]


CITY_ROUTE = [
    "Shanghai",
    "Beijing",
    "Guangzhou",
    "Shenzhen",
    "Hangzhou",
    "Chengdu",
    "Wuhan",
    "Mandalay",
]


def write_products():
    with open("products.csv", "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow([
            "product_id",
            "name",
            "category",
            "color",
            "size",
            "material",
            "price",
            "stock",
            "description",
        ])
        writer.writerows(PRODUCTS)
```

```
def product_weights():
    weights = {}
    for product_id, name, category, *_ in PRODUCTS:
        base = {
            "Coffee Mug": 10,
            "Travel Mug": 7,
            "Gift Set": 5,
            "Eco Mug": 5,
            "Latte Mug": 4,
            "Espresso Cup": 4,
            "Custom Mug": 4,
            "Outdoor Mug": 3,
            "Tea Mug": 3,
            "Smart Mug": 2,
            "Soup Mug": 2,
        }.get(category, 2)
        if product_id in {"P001", "P002", "P003", "P004", "P005"}:
            base += 8
        weights[product_id] = base
    return weights


def choose_product(category_preference=None):
    product_by_id = {row[0]: row for row in PRODUCTS}
    weights = product_weights()
    if category_preference:
        for product in PRODUCTS:
            if product[2] == category_preference:
                weights[product[0]] += 8
    product_ids = list(weights)
    selected = random.choices(product_ids, weights=[weights[pid] for pid in product_ids],
k=1)[0]
    return product_by_id[selected]


def write_orders():
    customers = [
        (f"U{index:03d}", f"Customer {index}", f"customer{index:03d}@example.com")
        for index in range(1, 81)
    ]
    category_preferences = [
        "Coffee Mug",
        "Travel Mug",
        "Gift Set",
        "Eco Mug",
        "Latte Mug",
        "Espresso Cup",
        "Custom Mug",
        "Outdoor Mug",
        "Tea Mug",
        "Smart Mug",
    ]
    product_by_id = {row[0]: row for row in PRODUCTS}
    start_date = date(2026, 1, 10)
    statuses = ["Delivered", "Shipped", "Processing", "Pending", "Cancelled"]
    status_weights = [45, 24, 17, 10, 4]
    order_rows = []
    order_number = 1

    for customer_index, (user_id, name, email) in enumerate(customers):
        preference = category_preferences[customer_index % len(category_preferences)]
        order_count = random.randint(2, 6)
        for _ in range(order_count):
            basket_size = random.choices([1, 2, 3, 4], weights=[35, 40, 18, 7], k=1)[0]
            basket = []
            if random.random() < 0.18:
                basket = [product_by_id["P001"], product_by_id["P002"], product_by_id["P003"]]
            elif random.random() < 0.15:
                basket = [product_by_id["P004"], product_by_id["P011"]]
            elif random.random() < 0.12:
                basket = [product_by_id["P005"], product_by_id["P031"]]
            while len(basket) < basket_size:
                product = choose_product(preference)
                if product not in basket:
                    basket.append(product)

            status = random.choices(statuses, weights=status_weights, k=1)[0]
            current_location = random.choice(CITY_ROUTE)
            address = random.choice(CITY_ROUTE)
            expected_date = start_date + timedelta(days=random.randint(2, 90))
            if status == "Cancelled":
                expected_delivery = "N/A"
```

```
            else:
                expected_delivery = expected_date.strftime("%d/%m/%Y")

            for product in basket:
                product_id, product_name, _, _, _, _, price, *_ = product
                quantity = random.choices([1, 2, 3, 4], weights=[55, 28, 12, 5], k=1)[0]
                total = round(price * quantity, 2)
                order_rows.append([
                    f"O_N{order_number:03d}",
                    status,
                    current_location,
                    expected_delivery,
                    address,
                    name,
                    email,
                    product_id,
                    product_name,
                    quantity,
                    f"{price:.2f}",
                    f"{total:.2f}",
                ])
                order_number += 1

    with open("orders.csv", "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow([
            "order_number",
            "status",
            "current_location",
            "expected_delivery_date",
            "customer_address",
            "customer_name",
            "customer_email",
            "product_id",
            "product_name",
            "quantity",
            "unit_price",
            "total_price",
        ])
        writer.writerows(order_rows)


def write_user_interactions():
    product_ids = [row[0] for row in PRODUCTS]
    rows = []
    start = date(2026, 1, 1)
    actions = ["view", "cart", "purchase", "review"]
    action_weights = [48, 22, 22, 8]
    for index in range(1, 701):
        user_id = f"U{random.randint(1, 80):03d}"
        product_id = random.choice(product_ids)
        action = random.choices(actions, weights=action_weights, k=1)[0]
        rating = ""
        if action in {"purchase", "review"}:
            rating = random.choices([3, 4, 5], weights=[14, 38, 48], k=1)[0]
        timestamp = start + timedelta(days=random.randint(0, 115))
        rows.append([user_id, product_id, action, rating, timestamp.isoformat()])

    with open("user_interactions.csv", "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(["user_id", "product_id", "action", "rating", "timestamp"])
        writer.writerows(rows)


def write_survey():
    comments = [
        "Responses were clear and recommendations were useful",
        "Product suggestions were relevant",
        "Analytics answers were easy to understand",
        "Order and product features worked well",
        "Recommendation feature could include more products",
        "The chatbot was fast and easy to use",
        "Market basket results helped explain related products",
        "The product category prediction was understandable",
    ]
    with open("evaluation_survey.csv", "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow([
            "respondent_id",
            "chatbot_accuracy",
            "rating_recommendations",
            "rating_speed",
```

```
            "rating_overall",
            "comments",
        ])
        for index in range(1, 31):
            writer.writerow([
                f"S{index:03d}",
                random.choices([3, 4, 5], weights=[8, 44, 48], k=1)[0],
                random.choices([3, 4, 5], weights=[12, 46, 42], k=1)[0],
                random.choices([3, 4, 5], weights=[5, 35, 60], k=1)[0],
                random.choices([3, 4, 5], weights=[8, 42, 50], k=1)[0],
                random.choice(comments),
            ])


if __name__ == "__main__":
    write_products()
    write_orders()
    write_user_interactions()
    write_survey()
    print("Generated products.csv, orders.csv, user_interactions.csv, and
evaluation_survey.csv")
```

app.py

```
import tkinter as tk
from tkinter import font
from tkinter import simpledialog

from chatbot_features import (
    category_prediction_message,
    market_basket_message,
    product_popularity_message,
    recommendation_message,
)
from chatbot_engine import (
    CONFIDENCE_THRESHOLD,
    PRODUCT_QUESTION_TAGS,
    ChatbotEngine,
    is_admin_login_request,
    is_admin_logout_request,
    is_cancel_order_request,
    is_category_prediction_request,
    is_create_order_request,
    is_market_basket_request,
    is_product_popularity_request,
    is_recommendation_request,
    is_update_status_request,
)
from db import (
    authenticate_user,
    create_order,
    delete_order,
    fetch_customer_address,
    fetch_order_info,
    init_db,
    seed_from_csv,
    update_address,
    update_order_status,
    verify_customer_email,
)
from product_catalog import (
    format_product,
    format_product_list,
    get_product_by_id,
    load_products,
)


class ChatGUI:
    def __init__(self, master):
        self.master = master
        master.title("E-commerce Chatbot")
        master.geometry("520x680")
        master.minsize(460, 560)
        master.configure(bg="#101827")
        master.option_add("*Font", "Helvetica 11")
        master.columnconfigure(0, weight=1)
        master.rowconfigure(1, weight=1)

        self.bot_name = "bot4urLIF3"
        self.current_user = None
        self.viewed_product_ids = set()
        self.cart_product_ids = []
        self.setup_styles()
        self.build_header()
        self.build_chat_area()
        self.build_input_area()

        self.engine = ChatbotEngine.from_files("data.pth", "intents.json")
        self.setup_database()
        self.display_message(
            self.bot_name,
            "Hi, I am bot4urLIF3. How can I help you today?",
        )

    def setup_styles(self):
        self.colors = {
            "app_bg": "#101827",
            "header_bg": "#162033",
            "panel": "#0b1220",
            "panel_border": "#263349",
            "text": "#e5edf8",
            "muted": "#91a3ba",
```

```
        "bot": "#6ea8fe",
        "user": "#5eead4",
        "bot_bubble": "#17233a",
        "user_bubble": "#123b3a",
        "button": "#3b82f6",
        "button_active": "#2563eb",
        "entry_bg": "#111c2f",
    }
    self.fonts = {
        "title": font.Font(family="Helvetica", size=18, weight="bold"),
        "subtitle": font.Font(family="Helvetica", size=10),
        "body": font.Font(family="Helvetica", size=12),
        "message": font.Font(family="Helvetica", size=12),
        "sender": font.Font(family="Helvetica", size=11, weight="bold"),
        "button": font.Font(family="Helvetica", size=11, weight="bold"),
    }

def build_header(self):
    header = tk.Frame(self.master, bg=self.colors["header_bg"])
    header.grid(row=0, column=0, sticky="ew", padx=16, pady=(16, 10))
    header.columnconfigure(0, weight=1)

    title = tk.Label(
        header,
        text="E-commerce Chatbot",
        bg=self.colors["header_bg"],
        fg=self.colors["text"],
        font=self.fonts["title"],
        anchor="w",
    )
    title.grid(row=0, column=0, sticky="w", padx=16, pady=(14, 0))

    subtitle = tk.Label(
        header,
        text="Customer support assistant",
        bg=self.colors["header_bg"],
        fg=self.colors["muted"],
        font=self.fonts["subtitle"],
        anchor="w",
    )
    subtitle.grid(row=1, column=0, sticky="w", padx=16, pady=(3, 14))

    status = tk.Label(
        header,
        text="Online",
        bg=self.colors["header_bg"],
        fg="#86efac",
        font=self.fonts["subtitle"],
    )
    status.grid(row=0, column=1, rowspan=2, sticky="e", padx=16)

def build_chat_area(self):
    chat_shell = tk.Frame(
        self.master,
        bg=self.colors["panel"],
        relief=tk.FLAT,
        highlightthickness=1,
        highlightbackground=self.colors["panel_border"],
        highlightcolor=self.colors["bot"],
    )
    chat_shell.grid(row=1, column=0, sticky="nsew", padx=16, pady=(0, 12))
    chat_shell.columnconfigure(0, weight=1)
    chat_shell.rowconfigure(0, weight=1)

    self.chat_canvas = tk.Canvas(
        chat_shell,
        bg=self.colors["panel"],
        bd=0,
        highlightthickness=0,
    )
    self.chat_canvas.grid(row=0, column=0, sticky="nsew")

    scrollbar = tk.Scrollbar(
        chat_shell,
        orient=tk.VERTICAL,
        command=self.chat_canvas.yview,
        relief=tk.FLAT,
        borderwidth=0,
    )
    scrollbar.grid(row=0, column=1, sticky="ns")
    scrollbar.configure(
        bg=self.colors["panel"],
```

```
        activebackground=self.colors["panel_border"],
        troughcolor=self.colors["panel"],
    )
    self.chat_canvas.configure(yscrollcommand=scrollbar.set)

    self.messages_frame = tk.Frame(self.chat_canvas, bg=self.colors["panel"])
    self.messages_window = self.chat_canvas.create_window(
        0,
        0,
        window=self.messages_frame,
        anchor="nw",
    )
    self.chat_canvas.bind("<Configure>", self.resize_chat_content)
    self.messages_frame.bind("<Configure>", self.update_chat_scroll_region)
    self.bind_mousewheel(self.chat_canvas)
    self.bind_mousewheel(self.messages_frame)

def resize_chat_content(self, event):
    self.chat_canvas.itemconfigure(self.messages_window, width=event.width)
    self.draw_chat_background(event.width, event.height)

def update_chat_scroll_region(self, _event=None):
    self.chat_canvas.configure(scrollregion=self.chat_canvas.bbox("all"))

def draw_chat_background(self, width, height):
    self.chat_canvas.delete("chat_bg")
    for x in range(-height, width, 96):
        self.chat_canvas.create_line(
            x,
            height,
            x + height,
            0,
            fill="#111c2f",
            width=1,
            tags="chat_bg",
        )
    for x in range(28, width, 96):
        for y in range(28, height, 96):
            self.chat_canvas.create_oval(
                x,
                y,
                x + 2,
                y + 2,
                fill="#1c2b45",
                outline="",
                tags="chat_bg",
            )
    self.chat_canvas.tag_lower("chat_bg")

def bind_mousewheel(self, widget):
    widget.bind("<MouseWheel>", self.on_chat_mousewheel)
    widget.bind("<Button-4>", self.on_chat_mousewheel)
    widget.bind("<Button-5>", self.on_chat_mousewheel)

def on_chat_mousewheel(self, event):
    if getattr(event, "num", None) == 4:
        self.chat_canvas.yview_scroll(-1, "units")
    elif getattr(event, "num", None) == 5:
        self.chat_canvas.yview_scroll(1, "units")
    else:
        direction = -1 if event.delta > 0 else 1
        self.chat_canvas.yview_scroll(direction, "units")

def build_input_area(self):
    input_frame = tk.Frame(self.master, bg=self.colors["app_bg"])
    input_frame.grid(row=2, column=0, sticky="ew", padx=16, pady=(0, 16))
    input_frame.columnconfigure(0, weight=1)

    self.input_field = tk.Entry(
        input_frame,
        bg=self.colors["entry_bg"],
        fg=self.colors["text"],
        insertbackground=self.colors["bot"],
        relief=tk.FLAT,
        borderwidth=0,
        highlightthickness=1,
        highlightbackground=self.colors["panel_border"],
        highlightcolor=self.colors["bot"],
        font=self.fonts["body"],
    )
    self.input_field.grid(row=0, column=0, sticky="ew", padx=(0, 10), ipady=11)
    self.input_field.bind("<Return>", self.send_message)
```

```
        self.send_button = tk.Label(
            input_frame,
            text="Send",
            bg=self.colors["button"],
            fg="#ffffff",
            cursor="hand2",
            font=self.fonts["button"],
            padx=22,
            pady=12,
        )
        self.send_button.grid(row=0, column=1)
        self.send_button.bind("<Button-1>", self.send_message)
        self.send_button.bind("<Enter>", self.on_send_hover)
        self.send_button.bind("<Leave>", self.on_send_leave)

    def on_send_hover(self, _event=None):
        self.send_button.configure(bg=self.colors["button_active"])

    def on_send_leave(self, _event=None):
        self.send_button.configure(bg=self.colors["button"])

    def setup_database(self):
        try:
            init_db()
            seed_from_csv("orders.csv")
        except Exception as e:
            print(f"Error initializing database: {e}")

    def send_message(self, event=None):
        user_input = self.input_field.get().strip()
        if not user_input:
            return
        self.input_field.delete(0, tk.END)
        self.display_message("You", user_input)
        self.generate_bot_response(user_input)

    def generate_bot_response(self, user_input):
        if is_admin_login_request(user_input):
            self.admin_login_gui()
            return
        if is_admin_logout_request(user_input):
            self.admin_logout()
            return
        if is_update_status_request(user_input):
            self.update_order_status_gui()
            return
        if is_cancel_order_request(user_input):
            self.get_response("cancel_order", 1.0)
            return
        if is_create_order_request(user_input):
            self.get_response("create_order", 1.0)
            return
        if is_recommendation_request(user_input):
            self.show_recommendations_gui(user_input)
            return
        if is_market_basket_request(user_input):
            self.show_market_basket_gui()
            return
        if is_product_popularity_request(user_input):
            self.show_product_popularity_gui()
            return
        if is_category_prediction_request(user_input):
            self.show_category_prediction_gui(user_input)
            return

        tag, prob = self.engine.predict(user_input)
        self.get_response(tag, prob, user_input)

    def is_admin(self):
        return self.current_user and self.current_user.get("role") == "admin"

    def get_response(self, tag, prob, user_input=None):
        if prob <= CONFIDENCE_THRESHOLD:
            self.display_message(
                self.bot_name,
                "I do not understand, please rephrase your question so I can better assist
you.",
            )
            return

        if tag == "recommendations":
```

```
        self.show_recommendations_gui(user_input or "")
        return

    self.display_message(self.bot_name, self.engine.get_response_text(tag))
    if tag == "order_status":
        self.track_order_gui()
    elif tag == "update_address":
        self.update_address_gui()
    elif tag == "create_order":
        self.create_order_gui()
    elif tag == "cancel_order":
        self.cancel_order_gui()
    elif tag == "admin_login":
        self.admin_login_gui()
    elif tag == "update_order_status":
        self.update_order_status_gui()
    elif tag in PRODUCT_QUESTION_TAGS:
        self.show_products_gui()

def display_message(self, sender, message):
    is_user = sender == "You"
    row = tk.Frame(self.messages_frame, bg=self.colors["panel"])
    row.pack(fill=tk.X, padx=18, pady=(14, 4))

    content = tk.Frame(row, bg=self.colors["panel"])
    content.pack(anchor="e" if is_user else "w")

    sender_label = tk.Label(
        content,
        text=sender,
        bg=self.colors["panel"],
        fg=self.colors["user"] if is_user else self.colors["bot"],
        font=self.fonts["sender"],
    )
    sender_label.pack(anchor="e" if is_user else "w", pady=(0, 4))

    bubble = tk.Label(
        content,
        text=message,
        bg=self.colors["user_bubble"] if is_user else self.colors["bot_bubble"],
        fg=self.colors["text"],
        font=self.fonts["message"],
        justify=tk.LEFT,
        anchor="w",
        padx=14,
        pady=10,
        wraplength=380,
    )
    bubble.pack(anchor="e" if is_user else "w")

    self.bind_mousewheel(row)
    self.bind_mousewheel(content)
    self.bind_mousewheel(sender_label)
    self.bind_mousewheel(bubble)
    self.master.after(20, self.scroll_chat_to_bottom)

def scroll_chat_to_bottom(self):
    self.update_chat_scroll_region()
    self.chat_canvas.yview_moveto(1.0)

def track_order_gui(self):
    order_number = simpledialog.askstring("Track Order", "Please enter your order number:")
    if order_number:
        order_info = fetch_order_info(order_number)
        if order_info:
            message = (
                f"Order Number: {order_info['order_number']}\n"
                f"Status: {order_info['status']}\n"
                f"Current Location: {order_info['current_location']}\n"
                f"Destination Address: {order_info['customer_address']}\n"
                f"Expected Delivery Date: {order_info['expected_delivery_date']}\n"
                f"Product: {order_info.get('product_name') or 'N/A'}\n"
                f"Quantity: {order_info.get('quantity') or 'N/A'}\n"
                f"Total Price: {order_info.get('total_price') or 'N/A'}\n"
            )
            self.display_message(self.bot_name, message)
        else:
            self.display_message(self.bot_name, "Order not found.")
    else:
        self.display_message(self.bot_name, "Order number cannot be empty.")

def update_address_gui(self):
```

```
        order_number = simpledialog.askstring(
            "Update Address",
            "Please enter your order number:",
        )
        email = simpledialog.askstring(
            "Update Address",
            "Please provide your email for verification:",
        )
        if order_number and email:
            if verify_customer_email(order_number, email):
                current_address = fetch_customer_address(email)
                new_address = simpledialog.askstring(
                    "Update Address",
                    f"Your current address is {current_address}. Please enter your new
address:",
                )
                if new_address:
                    confirmation = simpledialog.askstring(
                        "Update Address",
                        (
                            f"You entered the following address: {new_address}. "
                            "Are you sure this is your new address? (yes/no):"
                        ),
                    )
                    if confirmation.lower() == "yes":
                        update_address(order_number, new_address)
                        self.display_message(self.bot_name, "Your address has been updated.")
                    else:
                        self.display_message(self.bot_name, "Address update cancelled.")
            else:
                self.display_message(self.bot_name, "Email verification failed.")
        else:
            self.display_message(self.bot_name, "Order number and email cannot be empty.")

    def show_products_gui(self):
        products = load_products()
        self.viewed_product_ids.update(product["product_id"] for product in products)
        self.display_message(
            self.bot_name,
            "Available products:\n" + format_product_list(products),
        )

    def show_recommendations_gui(self, user_input):
        self.display_message(
            self.bot_name,
            recommendation_message(
                user_input,
                self.viewed_product_ids,
                self.cart_product_ids,
            ),
        )

    def show_product_popularity_gui(self):
        self.display_message(
            self.bot_name,
            product_popularity_message(),
        )

    def show_market_basket_gui(self):
        self.display_message(
            self.bot_name,
            market_basket_message(),
        )

    def show_category_prediction_gui(self, user_input):
        self.display_message(
            self.bot_name,
            category_prediction_message(user_input),
        )

    def select_product_gui(self):
        products = load_products()
        if not products:
            self.display_message(self.bot_name, "No products are available right now.")
            return None

        self.viewed_product_ids.update(product["product_id"] for product in products)
        self.display_message(
            self.bot_name,
            "Available products:\n" + format_product_list(products),
        )
        product_id = simpledialog.askstring(
```

```
        "Create Order",
        "Enter the product ID for this order:",
    )
    product = get_product_by_id(product_id)
    if not product:
        self.display_message(self.bot_name, "Product not found.")
        return None
    self.viewed_product_ids.add(product["product_id"])
    return product

def admin_login_gui(self):
    username = simpledialog.askstring("Admin Login", "Username:")
    password = simpledialog.askstring("Admin Login", "Password:", show="*")

    if not username or not password:
        self.display_message(self.bot_name, "Username and password are required.")
        return

    user = authenticate_user(username, password)
    if not user or user.get("role") != "admin":
        self.display_message(self.bot_name, "Admin login failed.")
        return

    self.current_user = user
    self.display_message(
        self.bot_name,
        f"Admin login successful. Welcome, {user.get('full_name') or username}.",
    )

def admin_logout(self):
    if not self.current_user:
        self.display_message(self.bot_name, "No admin is logged in.")
        return
    self.current_user = None
    self.display_message(self.bot_name, "Admin logged out.")

def update_order_status_gui(self):
    if not self.is_admin():
        self.display_message(
            self.bot_name,
            "Only an admin can update order status. Type 'admin login' first.",
        )
        return

    order_number = simpledialog.askstring(
        "Update Order Status",
        "Enter order number:",
    )
    status = simpledialog.askstring(
        "Update Order Status",
        "Enter new status (Processing, Shipped, Delivered, Cancelled):",
    )
    current_location = simpledialog.askstring(
        "Update Order Status",
        "Enter current location, or leave blank to keep existing:",
    )

    if not order_number or not status:
        self.display_message(self.bot_name, "Order number and status are required.")
        return

    if update_order_status(order_number, status, current_location or None):
        self.display_message(self.bot_name, "Order status updated successfully.")
    else:
        self.display_message(self.bot_name, "Order not found.")

def create_order_gui(self, selected_product=None):
    product = selected_product or self.select_product_gui()
    if not product:
        return

    customer_name = simpledialog.askstring("Create Order", "Enter customer name:")
    customer_email = simpledialog.askstring("Create Order", "Enter customer email:")
    customer_address = simpledialog.askstring("Create Order", "Enter customer address:")
    quantity = simpledialog.askinteger(
        "Create Order",
        "Enter quantity:",
        minvalue=1,
        initialvalue=1,
    )
    expected_delivery_date = simpledialog.askstring(
        "Create Order",
```

```
        "Enter expected delivery date (dd/mm/yyyy) or leave blank:",
    )

    if (
        not customer_name
        or not customer_email
        or not customer_address
        or not quantity
    ):
        self.display_message(
            self.bot_name,
            "Customer name, email, address, and quantity are required.",
        )
        return

    try:
        unit_price = float(product["price"])
        total_price = unit_price * quantity
        ok, message = create_order(
            customer_email=customer_email,
            customer_address=customer_address,
            customer_name=customer_name,
            status="Processing",
            current_location="Warehouse",
            expected_delivery_date=expected_delivery_date,
            product_id=product["product_id"],
            product_name=product["name"],
            quantity=quantity,
            unit_price=unit_price,
            total_price=total_price,
        )
        if ok:
            self.cart_product_ids.append(product["product_id"])
            self.display_message(
                self.bot_name,
                (
                    f"{message}\n"
                    f"Product: {format_product(product)}\n"
                    f"Quantity: {quantity}\n"
                    f"Total: ${total_price:.2f}"
                ),
            )
        else:
            self.display_message(self.bot_name, message)
    except Exception as e:
        self.display_message(self.bot_name, f"Failed to create order: {e}")

def cancel_order_gui(self):
    order_number = simpledialog.askstring(
        "Cancel Order",
        "Please enter the order number to cancel:",
    )
    email = simpledialog.askstring(
        "Cancel Order",
        "Please provide your email for verification:",
    )

    if not order_number or not email:
        self.display_message(self.bot_name, "Order number and email are required.")
        return

    if not verify_customer_email(order_number, email):
        self.display_message(self.bot_name, "Email verification failed.")
        return

    confirmation = simpledialog.askstring(
        "Cancel Order",
        f"Are you sure you want to cancel order {order_number}? (yes/no):",
    )
    if not confirmation or confirmation.lower() != "yes":
        self.display_message(self.bot_name, "Order cancellation cancelled.")
        return

    if delete_order(order_number):
        self.display_message(
            self.bot_name,
            "Your order has been cancelled and deleted from the database.",
        )
    else:
        self.display_message(self.bot_name, "Order not found.")
```

```
if __name__ == "__main__":
    root = tk.Tk()
    chat_gui = ChatGUI(root)
    root.mainloop()
```

chat.py

```
from chatbot_features import (
    category_prediction_message,
    market_basket_message,
    product_popularity_message,
    recommendation_message,
)
from chatbot_engine import (
    CONFIDENCE_THRESHOLD,
    PRODUCT_QUESTION_TAGS,
    ChatbotEngine,
    is_admin_login_request,
    is_admin_logout_request,
    is_cancel_order_request,
    is_category_prediction_request,
    is_create_order_request,
    is_market_basket_request,
    is_product_popularity_request,
    is_recommendation_request,
    is_update_status_request,
)
from db import (
    authenticate_user,
    create_order,
    delete_order,
    fetch_order_info,
    init_db,
    seed_from_csv,
    update_address,
    update_order_status,
    verify_customer_email,
)
from product_catalog import (
    format_product,
    format_product_list,
    get_product_by_id,
    load_products,
)


bot_name = "Lyla"


def get_user_input():
    return input("You: ").lower()


def handle_exit_commands(sentence, bot_name):
    exit_commands = ["quit", "exit", "bye", "thanks", "that’s all thanks", "goodbye"]
    if sentence in exit_commands:
        print(f"{bot_name}: Goodbye! Have a great day!")
        return True
    return False


def handle_special_cases(sentence, bot_name):
    special_cases = {
        "I don’t understand": "I apologize for any confusion. Please feel free to ask again.",
        "I have to go": "Alright! Take care and see you soon!",
    }
    if sentence in special_cases:
        print(f"{bot_name}: {special_cases[sentence]}")
        return True
    return False


def get_response(
    sentence,
    tag,
    prob,
    engine,
    bot_name,
    current_user=None,
    context=None,
    viewed_product_ids=None,
    cart_product_ids=None,
):
    if prob <= CONFIDENCE_THRESHOLD:
        print(f"{bot_name}: I do not understand...")
        return
```

```
    if tag == "recommendations":
        show_recommendations_cli(sentence, viewed_product_ids, cart_product_ids)
        return current_user

    print(f"{bot_name}: {engine.get_response_text(tag, context)}")
    if tag == "order_status":
        order_number = input("Please enter your order number: ")
        track_order(order_number)
    elif tag == "update_address":
        update_address_cli()
    elif tag == "create_order":
        create_order_cli(
            viewed_product_ids=viewed_product_ids,
            cart_product_ids=cart_product_ids,
        )
    elif tag == "cancel_order":
        cancel_order_cli()
    elif tag == "admin_login":
        return admin_login_cli()
    elif tag == "update_order_status":
        update_order_status_cli(current_user)
    elif tag in PRODUCT_QUESTION_TAGS:
        show_products_cli(viewed_product_ids)

    return current_user


def update_address_cli():
    email = input("Please provide your email for verification: ")
    order_number = input("Please enter your order number: ")

    if not verify_customer_email(order_number, email):
        print(f"{bot_name}: Email verification failed.")
        return

    current_address = fetch_order_info(order_number)['customer_address']
    print(
        f"{bot_name}: Your current registered address is: "
        f"{current_address}"
    )

    new_address = input("Please enter your new address: ")
    print(f"{bot_name}: You entered the following address: {new_address}")
    confirmation = input("Are you sure this is your new address? (yes/no): ")

    if confirmation.lower() == "yes":
        update_address(order_number, new_address)
        print(f"{bot_name}: Your address has been updated.")
    else:
        print(f"{bot_name}: Address update cancelled.")


def track_order(order_number):
    order_info = fetch_order_info(order_number)
    if order_info:
        print("")
        print("Order Information:")
        print(f"Order Number: {order_info['order_number']}")
        print(f"Status: {order_info['status']}")
        print(f"Current Location: {order_info['current_location']}")
        print(f"Destination Address: {order_info['customer_address']}")
        print(f"Expected Delivery Date: {order_info['expected_delivery_date']}")
        print(f"Product: {order_info.get('product_name') or 'N/A'}")
        print(f"Quantity: {order_info.get('quantity') or 'N/A'}")
        print(f"Total Price: {order_info.get('total_price') or 'N/A'}")
        print("")
    else:
        print("Order not found.")


def show_products_cli(viewed_product_ids=None):
    products = load_products()
    if viewed_product_ids is not None:
        viewed_product_ids.update(product["product_id"] for product in products)
    print("")
    print("Available Products:")
    print(format_product_list(products))
    print("")


def select_product_cli(viewed_product_ids=None):
```

```
    products = load_products()
    if not products:
        print(f"{bot_name}: No products are available right now.")
        return None

    if viewed_product_ids is not None:
        viewed_product_ids.update(product["product_id"] for product in products)
    print("")
    print("Available Products:")
    print(format_product_list(products))
    print("")
    product_id = input("Enter the product ID for this order: ").strip()
    product = get_product_by_id(product_id)
    if not product:
        print(f"{bot_name}: Product not found.")
        return None
    if viewed_product_ids is not None:
        viewed_product_ids.add(product["product_id"])
    return product


def create_order_cli(
    selected_product=None,
    viewed_product_ids=None,
    cart_product_ids=None,
):
    product = selected_product or select_product_cli(viewed_product_ids)
    if not product:
        return

    customer_name = input("Enter customer name: ").strip()
    customer_email = input("Enter customer email: ").strip()
    customer_address = input("Enter customer address: ").strip()
    quantity_input = input("Enter quantity: ").strip()
    expected_delivery_date = input(
        "Enter expected delivery date (dd/mm/yyyy) or leave blank: "
    ).strip()

    try:
        quantity = int(quantity_input)
    except ValueError:
        print("Quantity must be a number.")
        return

    if (
        not customer_name
        or not customer_email
        or not customer_address
        or quantity < 1
    ):
        print("Customer name, email, address, and quantity are required.")
        return

    try:
        unit_price = float(product["price"])
        total_price = unit_price * quantity
        ok, message = create_order(
            customer_email=customer_email,
            customer_address=customer_address,
            customer_name=customer_name,
            status="Processing",
            current_location="Warehouse",
            expected_delivery_date=expected_delivery_date or None,
            product_id=product["product_id"],
            product_name=product["name"],
            quantity=quantity,
            unit_price=unit_price,
            total_price=total_price,
        )
        if ok:
            if cart_product_ids is not None:
                cart_product_ids.append(product["product_id"])
            print(message)
            print(f"Product: {format_product(product)}")
            print(f"Quantity: {quantity}")
            print(f"Total: ${total_price:.2f}")
        else:
            print(message)
    except Exception as e:
        print(f"Failed to create order: {e}")
```

```
def show_recommendations_cli(
    sentence,
    viewed_product_ids=None,
    cart_product_ids=None,
):
    print("")
    print(f"{bot_name}: Here are personalized product recommendations.")
    print(recommendation_message(sentence, viewed_product_ids, cart_product_ids))
    print("")


def show_product_popularity_cli():
    print("")
    print(f"{bot_name}: Here is the product popularity analysis.")
    print(product_popularity_message())
    print("")


def show_market_basket_cli():
    print("")
    print(f"{bot_name}: Here is the market basket analysis.")
    print(market_basket_message())
    print("")


def show_category_prediction_cli(sentence):
    print("")
    print(f"{bot_name}: I used the product category prediction model.")
    print(category_prediction_message(sentence))
    print("")


def cancel_order_cli():
    order_number = input("Please enter the order number to cancel: ").strip()
    email = input("Please provide your email for verification: ").strip()

    if not order_number or not email:
        print(f"{bot_name}: Order number and email are required.")
        return

    if not verify_customer_email(order_number, email):
        print(f"{bot_name}: Email verification failed.")
        return

    confirmation = input(
        f"Are you sure you want to cancel order {order_number}? (yes/no): "
    )
    if confirmation.lower() != "yes":
        print(f"{bot_name}: Order cancellation cancelled.")
        return

    if delete_order(order_number):
        print(f"{bot_name}: Your order has been cancelled and deleted from the database.")
    else:
        print(f"{bot_name}: Order not found.")


def admin_login_cli():
    username = input("Username: ").strip()
    password = input("Password: ").strip()

    if not username or not password:
        print(f"{bot_name}: Username and password are required.")
        return None

    user = authenticate_user(username, password)
    if not user or user.get("role") != "admin":
        print(f"{bot_name}: Admin login failed.")
        return None

    print(f"{bot_name}: Admin login successful. Welcome, {user.get('full_name') or username}.")
    return user


def admin_logout_cli(current_user):
    if not current_user:
        print(f"{bot_name}: No admin is logged in.")
        return None

    print(f"{bot_name}: Admin logged out.")
    return None
```

```
def update_order_status_cli(current_user):
    if not current_user or current_user.get("role") != "admin":
        print(f"{bot_name}: Only an admin can update order status. Type ’admin login’ first.")
        return

    order_number = input("Enter order number: ").strip()
    status = input("Enter new status (Processing, Shipped, Delivered, Cancelled): ").strip()
    current_location = input("Enter current location, or leave blank to keep existing:
").strip()

    if not order_number or not status:
        print(f"{bot_name}: Order number and status are required.")
        return

    if update_order_status(order_number, status, current_location or None):
        print(f"{bot_name}: Order status updated successfully.")
    else:
        print(f"{bot_name}: Order not found.")


if __name__ == "__main__":
    try:
        init_db()
        seed_from_csv("orders.csv")
    except Exception as e:
        print(f"Error initializing database: {e}")

    try:
        engine = ChatbotEngine.from_files("data.pth", "intents.json")
    except Exception as e:
        print(f"Error loading chatbot model: {e}")
        raise SystemExit

    bot_name = "Lyla"
    context = None
    current_user = None
    viewed_product_ids = set()
    cart_product_ids = []

    print(
        "Hello! I am Lyla from CoffeeMugs. What do you need? "
        "(type ’quit’, ’exit’, or ’bye’ to exit)"
    )

    while True:
        sentence = get_user_input()

        if handle_exit_commands(sentence, bot_name) or handle_special_cases(sentence, bot_name):
            break

        if is_admin_login_request(sentence):
            tag, prob = "admin_login", 1.0
        elif is_admin_logout_request(sentence):
            current_user = admin_logout_cli(current_user)
            continue
        elif is_update_status_request(sentence):
            tag, prob = "update_order_status", 1.0
        elif is_cancel_order_request(sentence):
            tag, prob = "cancel_order", 1.0
        elif is_create_order_request(sentence):
            tag, prob = "create_order", 1.0
        elif is_recommendation_request(sentence):
            show_recommendations_cli(sentence, viewed_product_ids, cart_product_ids)
            continue
        elif is_market_basket_request(sentence):
            show_market_basket_cli()
            continue
        elif is_product_popularity_request(sentence):
            show_product_popularity_cli()
            continue
        elif is_category_prediction_request(sentence):
            show_category_prediction_cli(sentence)
            continue
        else:
            tag, prob = engine.predict(sentence)
        current_user = get_response(
            sentence,
            tag,
            prob,
            engine,
            bot_name,
```

```
    current_user,
    context,
    viewed_product_ids,
    cart_product_ids,
)
```

db.py

```
import csv
import hashlib
import hmac
import os
import re
from datetime import datetime
import pyodbc


DEFAULT_DRIVER = "ODBC Driver 18 for SQL Server"
DEFAULT_ADMIN_USERNAME = "admin"
DEFAULT_ADMIN_PASSWORD = "admin123"
DEFAULT_ADMIN_ROLE = "admin"
ORDER_NUMBER_PREFIX = "O_N"
ORDER_NUMBER_DIGITS = 3


def _normalize_trust_cert(conn_str):
    normalized = conn_str.replace(
        "TrustServerCertificate=True",
        "TrustServerCertificate=yes",
    )
    normalized = normalized.replace(
        "TrustServerCertificate=False",
        "TrustServerCertificate=no",
    )
    return normalized


def _ensure_encrypt(conn_str):
    if "Encrypt=" in conn_str or "ENCRYPT=" in conn_str:
        return conn_str
    return f"{conn_str};Encrypt=yes"


def _normalize_server(conn_str):
    if "Server=localhost" in conn_str and "Server=localhost," not in conn_str:
        return conn_str.replace("Server=localhost", "Server=127.0.0.1,1433")
    return conn_str


def _parse_conn_str(raw_conn_str):
    parts = [p for p in raw_conn_str.split(";") if p.strip()]
    items = []
    for part in parts:
        if "=" in part:
            key, value = part.split("=", 1)
            items.append((key.strip(), value.strip()))
        else:
            items.append((part.strip(), ""))
    return items


def _normalize_credentials(raw_conn_str):
    items = _parse_conn_str(raw_conn_str)
    normalized = []
    for key, value in items:
        key_lower = key.lower()
        if key_lower in {"user id", "userid", "uid"}:
            normalized.append(("UID", value))
        elif key_lower in {"password", "pwd"}:
            normalized.append(("PWD", value))
        else:
            normalized.append((key, value))
    return ";".join(
        f"{k}={v}" if v != "" else k for k, v in normalized
    )


def _build_odbc_conn_str(raw_conn_str):
    raw_conn_str = raw_conn_str.strip()
    raw_conn_str = _normalize_trust_cert(raw_conn_str)
    raw_conn_str = _ensure_encrypt(raw_conn_str)
    raw_conn_str = _normalize_server(raw_conn_str)
    raw_conn_str = _normalize_credentials(raw_conn_str)
    if "Driver=" in raw_conn_str or "DRIVER=" in raw_conn_str:
        return raw_conn_str
    return f"Driver={{{DEFAULT_DRIVER}}};{raw_conn_str}"
```

```
def get_connection():
    raw = os.getenv(
        "MSSQL_CONNECTION_STRING",
        (
            "Server=127.0.0.1,1433;Database=ChatbotDb;User Id=sa;"
            "Password=Ryen@123;TrustServerCertificate=yes;Encrypt=yes"
        ),
    )
    conn_str = _build_odbc_conn_str(raw)
    return pyodbc.connect(conn_str)


def init_db():
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            IF OBJECT_ID(N’dbo.orders’, N’U’) IS NULL
            BEGIN
                CREATE TABLE dbo.orders (
                    order_number NVARCHAR(50) NOT NULL PRIMARY KEY,
                    customer_email NVARCHAR(255) NOT NULL,
                    customer_address NVARCHAR(500) NOT NULL,
                    customer_name NVARCHAR(255) NULL,
                    status NVARCHAR(100) NOT NULL,
                    current_location NVARCHAR(255) NOT NULL,
                    expected_delivery_date DATE NULL,
                    product_id NVARCHAR(50) NULL,
                    product_name NVARCHAR(255) NULL,
                    quantity INT NOT NULL DEFAULT 1,
                    unit_price DECIMAL(10, 2) NULL,
                    total_price DECIMAL(10, 2) NULL
                );
            END
            """
        )
        cursor.execute(
            """
            IF COL_LENGTH(’dbo.orders’, ’customer_name’) IS NULL
            BEGIN
                ALTER TABLE dbo.orders ADD customer_name NVARCHAR(255) NULL;
            END
            """
        )
        cursor.execute(
            """
            IF EXISTS (
                SELECT 1
                FROM sys.columns
                WHERE object_id = OBJECT_ID(N’dbo.orders’)
                  AND name = ’expected_delivery_date’
                  AND is_nullable = 0
            )
            BEGIN
                ALTER TABLE dbo.orders ALTER COLUMN expected_delivery_date DATE NULL;
            END
            """
        )
        optional_columns = {
            "product_id": "NVARCHAR(50) NULL",
            "product_name": "NVARCHAR(255) NULL",
            "quantity": "INT NOT NULL CONSTRAINT DF_orders_quantity DEFAULT 1",
            "unit_price": "DECIMAL(10, 2) NULL",
            "total_price": "DECIMAL(10, 2) NULL",
        }
        for column, definition in optional_columns.items():
            cursor.execute(
                f"""
                IF COL_LENGTH(’dbo.orders’, ’{column}’) IS NULL
                BEGIN
                    ALTER TABLE dbo.orders ADD {column} {definition};
                END
                """
            )
        cursor.execute(
            """
            IF OBJECT_ID(N’dbo.users’, N’U’) IS NULL
            BEGIN
                CREATE TABLE dbo.users (
                    user_id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
                    username NVARCHAR(100) NOT NULL UNIQUE,
```

```
                    password_hash NVARCHAR(255) NOT NULL,
                    role NVARCHAR(50) NOT NULL,
                    full_name NVARCHAR(255) NULL,
                    email NVARCHAR(255) NULL,
                    address NVARCHAR(500) NULL
                );
            END
            """
        )
        conn.commit()
    seed_default_admin()


def hash_password(password):
    salt = os.urandom(16)
    password_hash = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt,
        100000,
    )
    return f"{salt.hex()}:{password_hash.hex()}"


def verify_password(password, stored_hash):
    try:
        salt_hex, hash_hex = stored_hash.split(":", 1)
        expected_hash = bytes.fromhex(hash_hex)
        actual_hash = hashlib.pbkdf2_hmac(
            "sha256",
            password.encode("utf-8"),
            bytes.fromhex(salt_hex),
            100000,
        )
        return hmac.compare_digest(actual_hash, expected_hash)
    except ValueError:
        return False


def seed_default_admin():
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT 1 FROM dbo.users WHERE username = ?;",
            DEFAULT_ADMIN_USERNAME,
        )
        if cursor.fetchone():
            return

        cursor.execute(
            """
            INSERT INTO dbo.users
                (username, password_hash, role, full_name, email)
            VALUES (?, ?, ?, ?, ?);
            """,
            DEFAULT_ADMIN_USERNAME,
            hash_password(DEFAULT_ADMIN_PASSWORD),
            DEFAULT_ADMIN_ROLE,
            "Default Admin",
            "admin@example.com",
        )
        conn.commit()


def authenticate_user(username, password):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT username, password_hash, role, full_name, email, address
            FROM dbo.users
            WHERE username = ?;
            """,
            username,
        )
        row = cursor.fetchone()
        if not row or not verify_password(password, row.password_hash):
            return None
        return {
            "username": row.username,
            "role": row.role,
            "full_name": row.full_name,
```

```
            "email": row.email,
            "address": row.address,
        }


def _is_table_empty():
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT COUNT(1) FROM dbo.orders;")
        count = cursor.fetchone()[0]
        return count == 0


def seed_from_csv(csv_path="orders.csv"):
    if not os.path.exists(csv_path):
        return
    if not _is_table_empty():
        return

    rows = []
    with open(csv_path, mode="r") as file:
        reader = csv.DictReader(file)
        for row in reader:
            parsed_date = _parse_order_date(row.get("expected_delivery_date"))
            rows.append(
                (
                    row["order_number"],
                    row["customer_email"],
                    row["customer_address"],
                    row.get("customer_name"),
                    row["status"],
                    row["current_location"],
                    parsed_date,
                    row.get("product_id"),
                    row.get("product_name"),
                    int(row.get("quantity") or 1),
                    row.get("unit_price") or None,
                    row.get("total_price") or None,
                )
            )

    if not rows:
        return

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.fast_executemany = True
        cursor.executemany(
            """
            INSERT INTO dbo.orders
                (
                    order_number,
                    customer_email,
                    customer_address,
                    customer_name,
                    status,
                    current_location,
                    expected_delivery_date,
                    product_id,
                    product_name,
                    quantity,
                    unit_price,
                    total_price
                )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
            """,
            rows,
        )
        conn.commit()


def _parse_order_date(raw_date):
    raw_date = (raw_date or "").strip()
    if not raw_date or raw_date.upper() == "N/A":
        return None
    return datetime.strptime(raw_date, "%d/%m/%Y").date()


def fetch_order_info(order_number):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
```

```
            """
            SELECT
                order_number,
                customer_email,
                customer_address,
                customer_name,
                status,
                current_location,
                expected_delivery_date,
                product_id,
                product_name,
                quantity,
                unit_price,
                total_price
            FROM dbo.orders
            WHERE order_number = ?;
            """,
            order_number,
        )
        row = cursor.fetchone()
        if not row:
            return None
        return {
            "order_number": row.order_number,
            "customer_email": row.customer_email,
            "customer_address": row.customer_address,
            "customer_name": getattr(row, "customer_name", None),
            "status": row.status,
            "current_location": row.current_location,
            "expected_delivery_date": str(row.expected_delivery_date),
            "product_id": getattr(row, "product_id", None),
            "product_name": getattr(row, "product_name", None),
            "quantity": getattr(row, "quantity", None),
            "unit_price": getattr(row, "unit_price", None),
            "total_price": getattr(row, "total_price", None),
        }


def verify_customer_email(order_number, customer_email):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT 1
            FROM dbo.orders
            WHERE order_number = ? AND customer_email = ?;
            """,
            order_number,
            customer_email,
        )
        return cursor.fetchone() is not None


def fetch_customer_address(email):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            SELECT customer_address
            FROM dbo.orders
            WHERE customer_email = ?;
            """,
            email,
        )
        row = cursor.fetchone()
        return row.customer_address if row else None


def update_address(order_number, new_address):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            UPDATE dbo.orders
            SET customer_address = ?
            WHERE order_number = ?;
            """,
            new_address,
            order_number,
        )
        conn.commit()
```

```
def update_order_status(order_number, status, current_location=None):
    with get_connection() as conn:
        cursor = conn.cursor()
        if current_location:
            cursor.execute(
                """
                UPDATE dbo.orders
                SET status = ?, current_location = ?
                WHERE order_number = ?;
                """,
                status,
                current_location,
                order_number,
            )
        else:
            cursor.execute(
                """
                UPDATE dbo.orders
                SET status = ?
                WHERE order_number = ?;
                """,
                status,
                order_number,
            )
        updated_count = cursor.rowcount
        conn.commit()
        return updated_count > 0


def delete_order(order_number):
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            DELETE FROM dbo.orders
            WHERE order_number = ?;
            """,
            order_number,
        )
        deleted_count = cursor.rowcount
        conn.commit()
        return deleted_count > 0


def generate_order_number():
    with get_connection() as conn:
        cursor = conn.cursor()
        return _generate_order_number(cursor)


def _generate_order_number(cursor):
    pattern = f"{ORDER_NUMBER_PREFIX}%"
    regex = re.compile(rf"^{re.escape(ORDER_NUMBER_PREFIX)}(\d+)$")

    cursor.execute(
        """
        SELECT order_number
        FROM dbo.orders
        WHERE order_number LIKE ?;
        """,
        pattern,
    )

    highest_number = 0
    for row in cursor.fetchall():
        match = regex.match(row.order_number)
        if match:
            highest_number = max(highest_number, int(match.group(1)))

    next_number = highest_number + 1
    return f"{ORDER_NUMBER_PREFIX}{next_number:0{ORDER_NUMBER_DIGITS}d}"


def create_order(
    customer_email,
    customer_address,
    status,
    current_location,
    expected_delivery_date=None,
    customer_name=None,
    product_id=None,
```

```
    product_name=None,
    quantity=1,
    unit_price=None,
    total_price=None,
    order_number=None,
):
    parsed_date = _parse_order_date(expected_delivery_date)
    with get_connection() as conn:
        cursor = conn.cursor()
        order_number = order_number or _generate_order_number(cursor)
        cursor.execute(
            "SELECT 1 FROM dbo.orders WHERE order_number = ?;",
            order_number,
        )
        if cursor.fetchone():
            return False, "Order number already exists."

        cursor.execute(
            """
            INSERT INTO dbo.orders
                (
                    order_number,
                    customer_email,
                    customer_address,
                    customer_name,
                    status,
                    current_location,
                    expected_delivery_date,
                    product_id,
                    product_name,
                    quantity,
                    unit_price,
                    total_price
                )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
            """,
            order_number,
            customer_email,
            customer_address,
            customer_name,
            status,
            current_location,
            parsed_date,
            product_id,
            product_name,
            quantity,
            unit_price,
            total_price,
        )
        conn.commit()
        return True, f"Order created. Order number: {order_number}"
```

train.py

```
import json

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset

from model import NeuralNet
from nltk_utils import bag_of_words, lemmatize, tokenize


def load_data(filename):
    with open(filename, 'r') as f:
        intents = json.load(f)

    all_words = []
    tags = []
    xy = []

    for intent in intents['intents']:
        tag = intent['tag']
        tags.append(tag)
        for pattern in intent['queries']:
            w = tokenize(pattern)
            all_words.extend(w)
            xy.append((w, tag))

    ignore_words = ['?', '.', '!', 'is', 'the', 'are', 'what', 'can']
    all_words = [lemmatize(w) for w in all_words if w not in ignore_words]
    all_words = sorted(set(all_words))
    tags = sorted(set(tags))

    return all_words, tags, xy


def create_training_data(all_words, tags, xy):
    X_train = []
    y_train = []

    for pattern_sentence, tag in xy:
        bag = bag_of_words(pattern_sentence, all_words)
        X_train.append(bag)
        label = tags.index(tag)
        y_train.append(label)

    return np.array(X_train), np.array(y_train)


class ChatDataset(Dataset):
    def __init__(self, X, y):
        self.x_data = torch.Tensor(X).float()
        self.y_data = torch.Tensor(y).long()
        self.n_samples = len(self.x_data)

    def __getitem__(self, index):
        return self.x_data[index], self.y_data[index]

    def __len__(self):
        return self.n_samples


def set_hyperparameters(X_train, tags):
    input_size = len(X_train[0])
    hidden_size = 16
    output_size = len(tags)
    num_epochs = 1000
    batch_size = 8
    learning_rate = 0.001

    return input_size, hidden_size, output_size, num_epochs, batch_size, learning_rate


def train_model(
    dataset,
    input_size,
    hidden_size,
    output_size,
    num_epochs,
    batch_size,
```

```
    learning_rate,
):
    train_loader = DataLoader(dataset=dataset, batch_size=batch_size, shuffle=True)
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = NeuralNet(input_size, hidden_size, output_size).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    for epoch in range(num_epochs):
        for words, labels in train_loader:
            words = words.to(device)
            labels = labels.to(device)

            outputs = model(words)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        if (epoch + 1) % 100 == 0:
            print(f'Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}')

    print(f'Final loss: {loss.item():.4f}')
    return model


def save_model(
    model,
    input_size,
    hidden_size,
    output_size,
    all_words,
    tags,
    filename,
):
    data = {
        "model_state": model.state_dict(),
        "input_size": input_size,
        "hidden_size": hidden_size,
        "output_size": output_size,
        "all_words": all_words,
        "tags": tags,
    }
    torch.save(data, filename)
    print(f'Training complete. Model saved to {filename}')


if __name__ == "__main__":
    all_words, tags, xy = load_data('intents.json')

    print(f"Number of queries: {len(xy)}")
    print(f"Number of tags: {len(tags)}")
    print("\nUnique lemmatized Words:")
    print(" ".join(all_words))

    X_train, y_train = create_training_data(all_words, tags, xy)
    (
        input_size,
        hidden_size,
        output_size,
        num_epochs,
        batch_size,
        learning_rate,
    ) = set_hyperparameters(X_train, tags)
    dataset = ChatDataset(X_train, y_train)
    model = train_model(
        dataset,
        input_size,
        hidden_size,
        output_size,
        num_epochs,
        batch_size,
        learning_rate,
    )
    save_model(
        model,
        input_size,
        hidden_size,
        output_size,
        all_words,
        tags,
```

```
    'data.pth',
)
```

model.py

```
import torch.nn as nn


class NeuralNet(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(NeuralNet, self).__init__()
        self.l1 = nn.Linear(input_size, hidden_size)
        self.l2 = nn.Linear(hidden_size, hidden_size)
        self.l3 = nn.Linear(hidden_size, num_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        out = self.l1(x)
        out = self.relu(out)
        out = self.l2(out)
        out = self.relu(out)
        out = self.l3(out)

        return out
```

nltk_utils.py

```
import numpy as np  # type: ignore[import-untyped]
import nltk  # type: ignore[import-untyped]
from nltk.stem.porter import PorterStemmer  # type: ignore[import-untyped]
from nltk.stem import WordNetLemmatizer  # type: ignore[import-untyped]

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()


def tokenize(sentence):
    """
    split sentence into array of words/tokens
    a token can be a word or punctuation character, or number
    """
    return nltk.word_tokenize(sentence)


def stem(word):
    """
    stemming = find the root form of the word
    examples:
    words = ["organize", "organizes", "organizing"]
    words = [stem(w) for w in words]
    -> ["organ", "organ", "organ"]
    """
    return stemmer.stem(word.lower())


def lemmatize(word):
    """
    lemmatization = get base form of the word
    examples:
    words = ["organize", "organizes", "organizing"]
    words = [lemmatize(w) for w in words]
    -> ["organize", "organize", "organizing"]
    """
    return lemmatizer.lemmatize(word.lower())


def bag_of_words(tokenized_sentence, words):
    """
    return bag of words array:
    1 for each known word that exists in the sentence, 0 otherwise
    example:
    sentence = ["hello", "how", "are", "you"]
    words = ["hi", "hello", "I", "you", "bye", "thank", "cool"]
    bog   = [  0 ,    1 ,    0 ,   1 ,    0 ,    0 ,      0]
    """
    sentence_words = [stem(word) for word in tokenized_sentence]

    bag = np.zeros(len(words), dtype=np.float32)
    for idx, w in enumerate(words):
        if w in sentence_words:
            bag[idx] = 1

    return bag
```